Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
Уральский технический институт связи и информатики (филиал) в г. Екатеринбурге
ЦК ИТ и АСУ кафедры ИСТ

Отчет
по учебной практике
по МДК 02.02: «Инструментальные средства разработки программного обеспечения»
на тему: «Разработка дашборда для расчета и визуализации KPI и управления доступом сотрудников и автотранспорта»

студента IV курса 283 группы
Фамилия Слесарев Имя,
отчество Владислав Эльдарович
Факультет Инфокоммуникаций, информатики и управления
Специальность 09.02.07 «Информационные системы и программирование»

г. Екатеринбург 2026 г.

Отзыв руководителя

Содержание

Введение 3
1 Вариант задания демоэкзамена 4
1.1 Краткая спецификация разрабатываемого модуля 4
1.2 Функциональные и нефункциональные требования 7
1.3 Обоснование выбора инструментальных средств разработки 8
1.4 Разработка интерфейса программного модуля 9
2 Разработка функционала дипломного работы 16
2.1 Доработка модуля корзины покупки 16
2.2 Разработка модуля отзывов о продуктах 17
2.3 Разработка модуля фермерской панели и управления продукцией 18
Заключение 20
Библиография 21
Приложение А 22

Ведение

Компания «АвтоТранс» — современный автосервис, специализирующийся на обслуживании и ремонте легковых и коммерческих автомобилей. В условиях растущей нагрузки и высоких требований клиентов необходима автоматизация бизнес-процессов: приём заявок, контроль сроков, распределение работ между механиками, учет выполненных заказов, обратная связь и статистика. Решаемый проект – это информационная система учета заявок и управления работой сервиса, объединяющая веб-интерфейс для заказчика, механика, менеджера и администратора, а также REST-API на backend.

Цель разработки: создать надежный и удобный программный модуль для «АвтоТранс», обеспечивающий полный цикл обработки заявки от регистрации до закрытия с поддержкой ролей, истории изменений, уведомлений и отчетности.

Для достижения цели определены задачи:

– спроектировать и реализовать базу данных заявок, пользователей и комментариев с привязкой ролей и связей (клиент, механик, менеджер);

– реализовать регистрацию и авторизацию (логин/пароль), хранение сессии/токена и разграничение доступа по ролям (ACL);

– реализовать CRUD-заявки: создание, просмотр, редактирование, изменение статуса, назначение/смену механика, назначение дополнительных механиков;

– обеспечить механизм запросов продления срока, согласования клиентом, автоматической корректировки даты и статусов;

–обеспечить возможность комментирования заявки механиком, фиксацию использованных материалов и статуса;

– разработать фронтенд (React) с интерфейсами;

– для менеджера (полный перечень заявок, фильтры, назначение, продление, отчеты);

– для механика (свои заявки, изменения статуса, комментарии);

– для клиента (свои заявки, создание, согласование продления, отзывы по QR);

– реализовать отчеты/статистику: количество заявок, среднее время выполнения, распределение по типам неисправностей, по мастерам;

– подготовить удобный пользовательский интерфейс (UX) с понятными уведомлениями, отзывчивым дизайном и возможностью работы на Windows (и иным хостинге через кроссплатформенность .NET).

В результате проект должен обеспечить «АвтоТранс» инструмент, снижающий ручной труд, уменьшающий ошибки передачи информации, ускоряющий циклы обслуживания и повышающий прозрачность работы сервисной службы.

1 Вариант задания демоэкзамена

1.1 Краткая спецификация разрабатываемого модуля

Модуль: учёт заявок на ремонт авто для сервиса «АвтоТранс».

Цель: автоматизация приема, обработки, статусов, комментариев и отчётности по заявкам.

Входные данные

создание заявки: данные клиента (ФИО, телефон), авто (марка, модель, тип), описание проблемы, плановая дата;

регистрация/авторизация пользователя: логин, пароль, роль (Администратор, Менеджер, Автомеханик, Клиент);

изменение заявки: статус (Новая, В работе, Готова к выдаче, Завершена), назначение мастера, продление сроков;

комментарии мастера (работы, материалы, замечания);

запрос доп. механика и продления;

параметры поиска: номер заявки, статус, клиент, тип авто, мастер.

Выходные данные

– список заявок (API /api/request, /api/request/{id});

– детальная страница заявки с историей комментариев, статусом, мастером, сроками;

– статистика (выполненные, среднее время, распределение по типам неисправностей);

– уведомления пользователю: успешное изменение статуса, назначение, продление, выдача;

– QR-код заявки (в т.ч. для обратной связи при завершении);

– результаты поиска пользователя.

Основной алгоритм (условный)

клиент/менеджер создаёт заявку.

заявка попадает в Новая.

менеджер/админ назначает мастера.

мастер переводит в В работе → Готова к выдаче → Завершена.

менеджер может запросить продление; клиент согласует или отклоняет.

система обновляет плановую дату, статус, хранит extensionStatus.

при Завершена генерируется QR (ссылка на форму оценки).

статистика обновляется для панели отчетов.

Рисунок 1.1: диаграмма процесса.

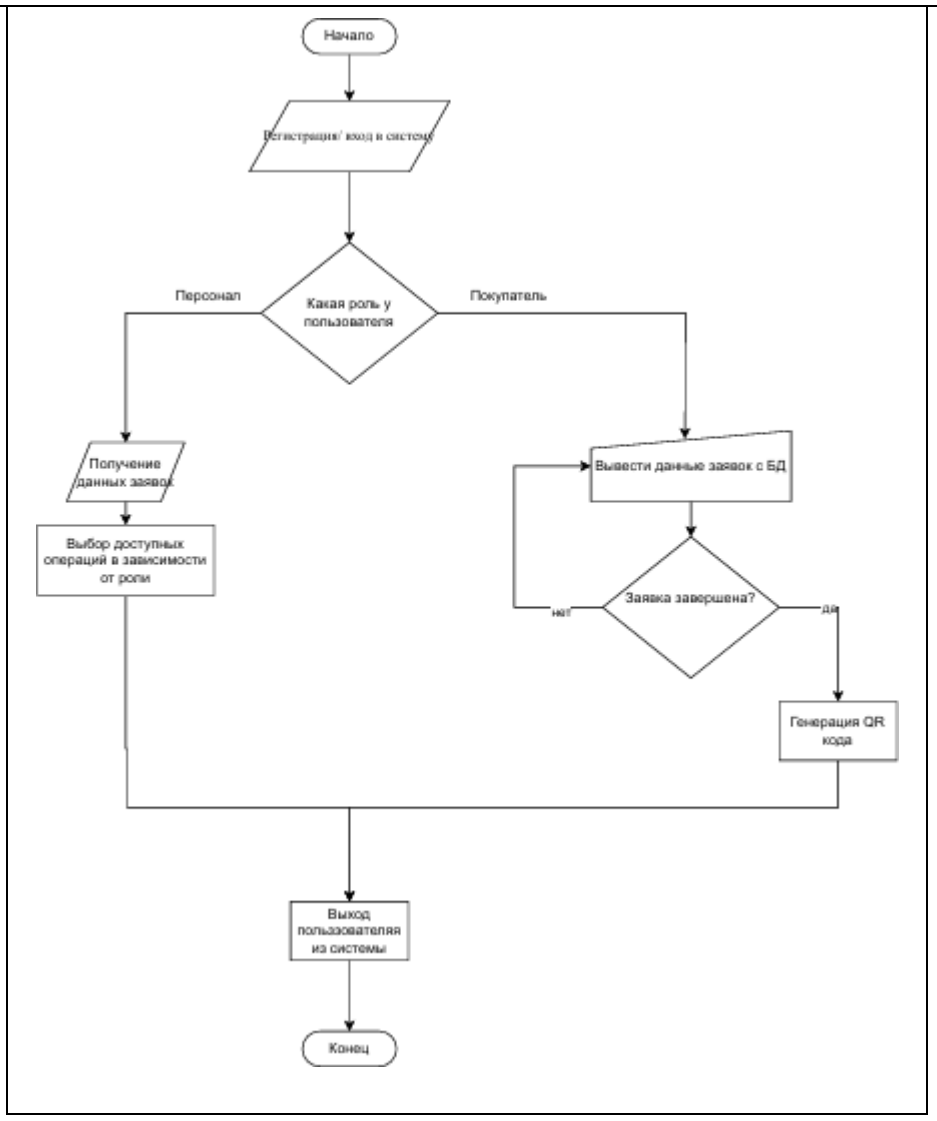

Рисунок 1.1 – Блок-схема основного алгоритма

ERD-диаграмма базы данных (Рисунок 1.2).

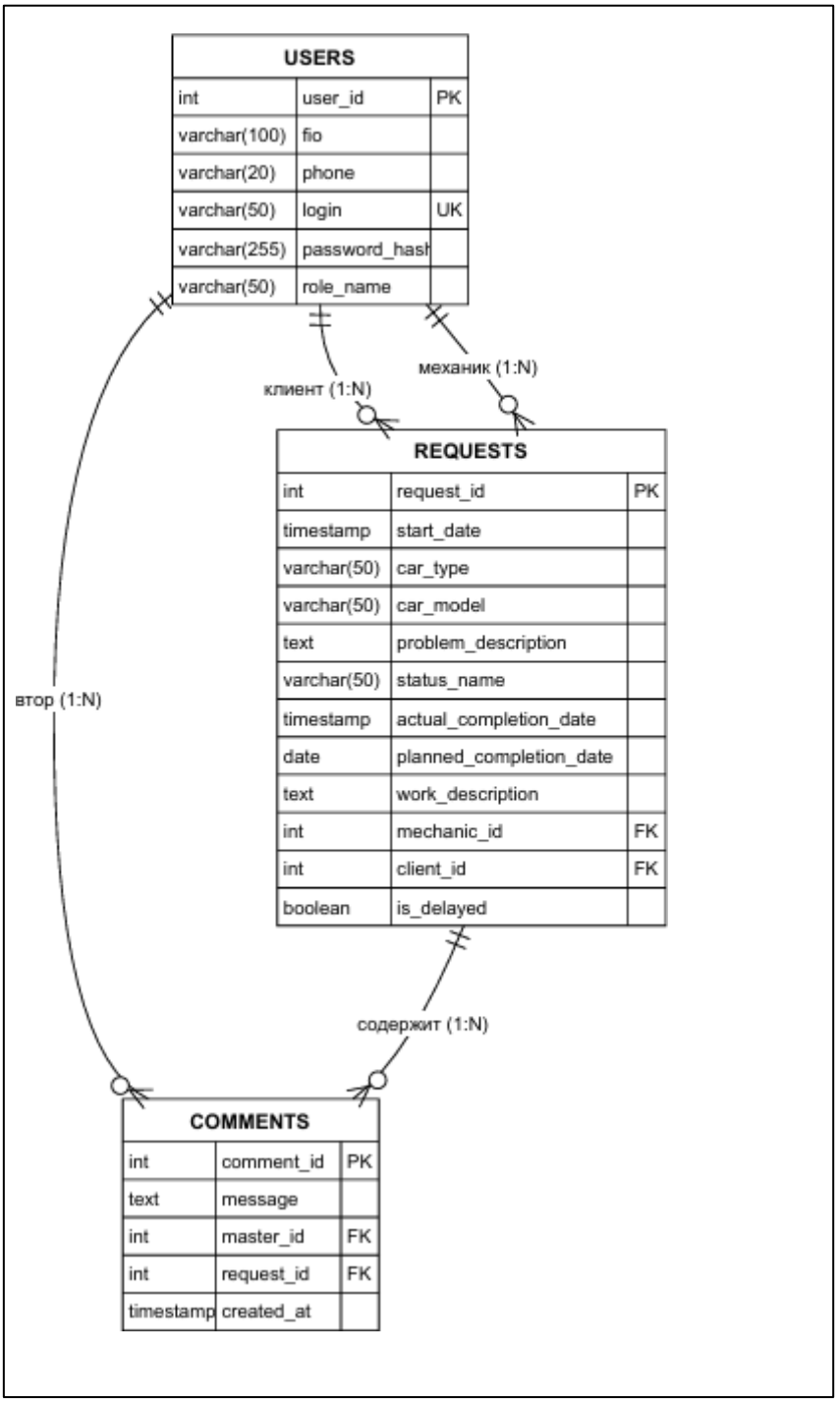

Рисунок 1.2 – ERD-диаграмма

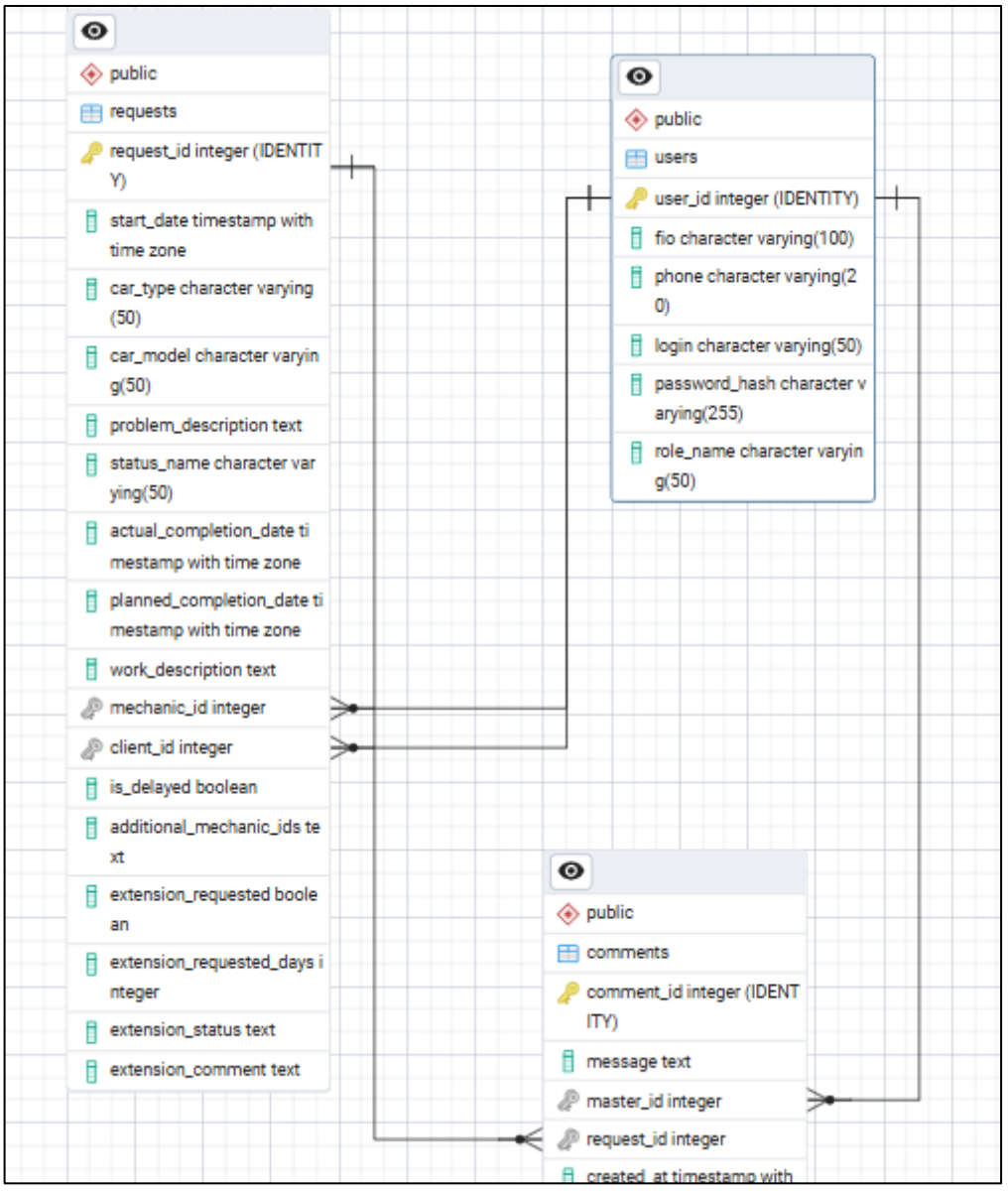

Рисунок 1.3 – ER-диаграмма(физическая модель)

1.2 Функциональные и нефункциональные требования

Функциональные требования

создание заявки с параметрами: RequestId, StartDate, CarType, CarModel, ProblemDescription, ClientId, Phone, RequestStatus.

редактирование заявки: изменение статуса (В работе, Готова к выдаче, Завершена), комментариев, назначенного мастера, плановой даты.

просмотр списка заявок (в Requests), загрузка подробной информации по заявке (/request/{id}).

поиск по номеру заявки.

поиск по параметрам: статус, модель авто, мастер, клиент.

статистика: число завершённых заявок, среднее время выполнения, распределение по типам проблем.

назначение автомеханика, снятие автомеханика, добавление дополнительного механика.

запрос продления срока и согласование продления с клиентом.

отслеживание состояния работы по статусу и полю isDelayed.

комментарии мастера к заявке (запчасти, работы, материалы).

генерация QR-кода для заявки (в том числе: при Завершена переход на форму обратной связи).

регистрация/авторизация пользователей; роли Администратор, Менеджер, Автомеханик, Клиент.

управление пользователями: смена роли, блокировка/активация.

Нефункциональные требования

поддержка Windows (основная платформа) и кроссплатформенность .NET (Linux поддержка на сервере).

понятный UI: React + Vite, простое меню, чёткие уведомления.

высокая скорость ответа благодаря Web API + EF Core + PostgreSQL.

надёжная авторизация/аутентификация; роли и route-protect.

безопасное обращение с данными (параметризованные запросы через EF Core).

масштабируемость: API + SPA разграничены, можно масштабировать независимо.

корректный UX через асинхронные обновления и межстраничные переходы.

1.3 Обоснование выбора инструментальных средств разработки

Язык и платформа: C# / .NET

надежность, типобезопасность, компиляционная проверка (бизнес-логика на Request, User, Comment);

производительность .NET 9 (сравнима с C++/Go, выдерживает нагрузку от запросов к /request и /auth);

богатый стек: EF Core (ORM), ASP.NET Web API, генерация QR (QRCoder), миграции или EnsureCreated;

кроссплатформенность (Windows + Linux);

встроенная безопасность (аутентификация, CORS, защита от SQL-инъекций, работа с JWT).

Среда разработки: Visual Studio 2022

мощная IDE для .NET: отладка, инспекция, рефакторинг, IntelliSense.

интеграция DB: PostgreSQL через расширения, работа с EF Core/миграциями.

ускоренная разработка контроллеров, моделей, DTO.

поддержка последних версий .NET/C# и unit-тестов.

Инструменты фронтенда

React + Vite (SPA):

- страницы: Requests, RequestDetails, UserManagement, Login, Register;
- компонент ProtectedRoute по ролям;
- Axios (services/api.js) для запросов, централизованное управление URL/token;
- CSS-модули (или .css как RequestDetails.css) для пользовательских стилей.

1.4 Разработка интерфейса программного модуля

При запуске приложения пользователь попадает на страницу Логин, приставленном Рисунке 1.4.
В форме: поля login и password, кнопка Войти, ссылка Зарегистрироваться.
Для теста:
администратор: admin / admin :
механик: master/ master;
клиент: client/ client.
После успешной авторизации:
– запрос на /api/auth/login;
– получение токена и роли;
– сохранение в localStorage;
– редирект на главную страницу по роли.

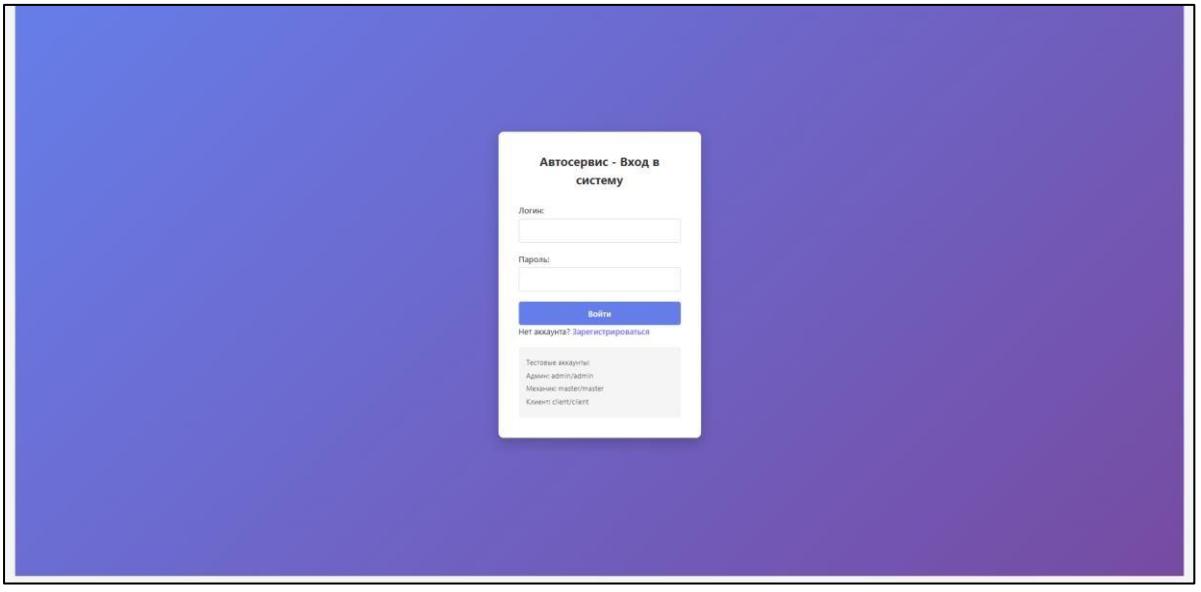

Рисунок 1.4 – Окно авторизации

Окно регистрации представлено на рисунке 1.5. Имеются поля для ввода ФИО, номера телефона, логина, пароля, а также для подтверждения пароля, кнопка «Зарегистрироваться». Для пользователей, имеющих авторизованный профиль, имеется ссылка на окно входа.

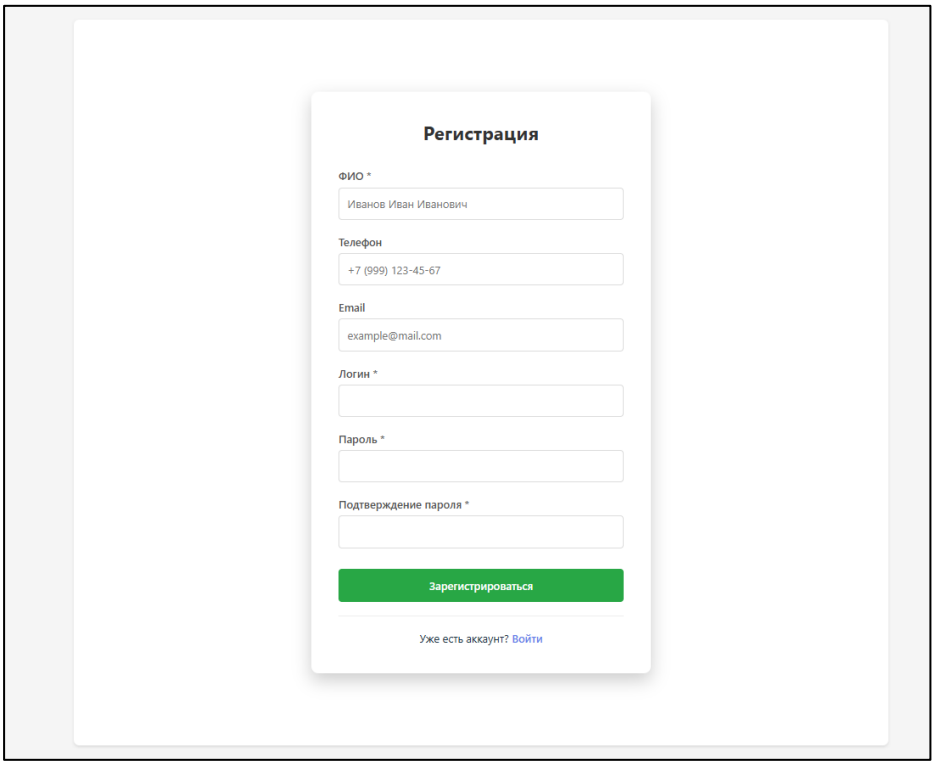

Рисунок 1.5 – Страница регистрации

Менеджер видит:

– список всех заявок (Requests);

– фильтры по номеру, клиенту, мастеру, статусу, дате;

– кнопки Заявки, Контроль качества;

– по строке — Просмотр, Редактировать.

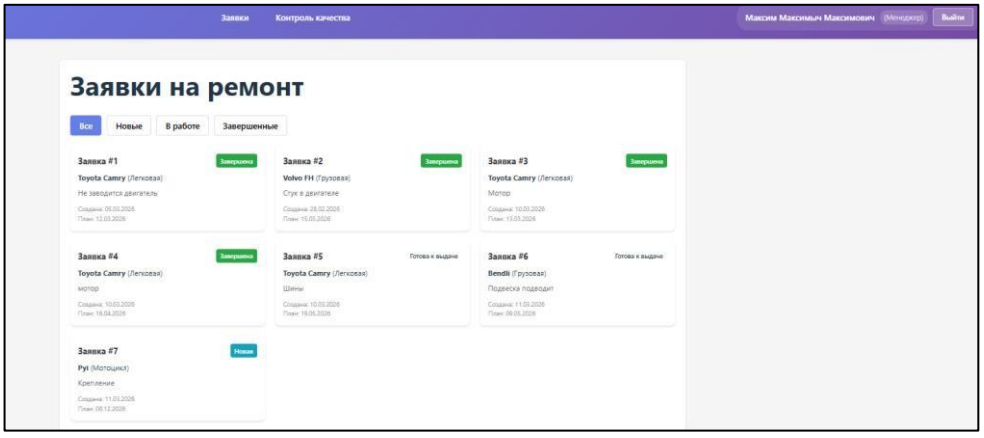

Рисунок 1.6– Окно просмотра

Окно редактирования заявки представлено на рисунке 1.7 (для пользователя «менеджер»). Имеется возможность продлевать сроки, устанавливать новую дату, отправлять запрос продления клиенту, добавлять дополнительных механиков, назначать или снимать мастеров. Дополнительно может указать комментарии.

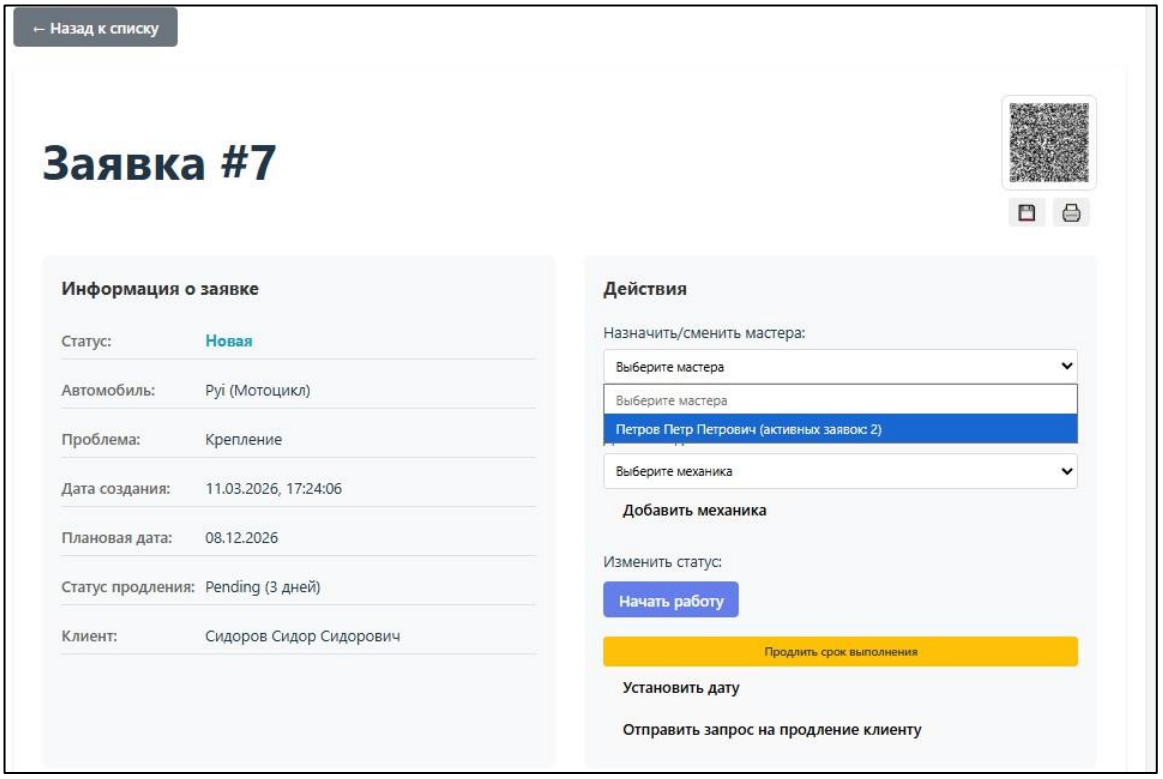

Рисунок 1.7 – Окно редактирования

На рисунке 1.8 представлено окно добавления новой заявки (для пользователя «клиент»). Имеются поля ввода модели автомобиля и описания проблемы, выпадающие списки для выбора вида автомобиля и клиента.

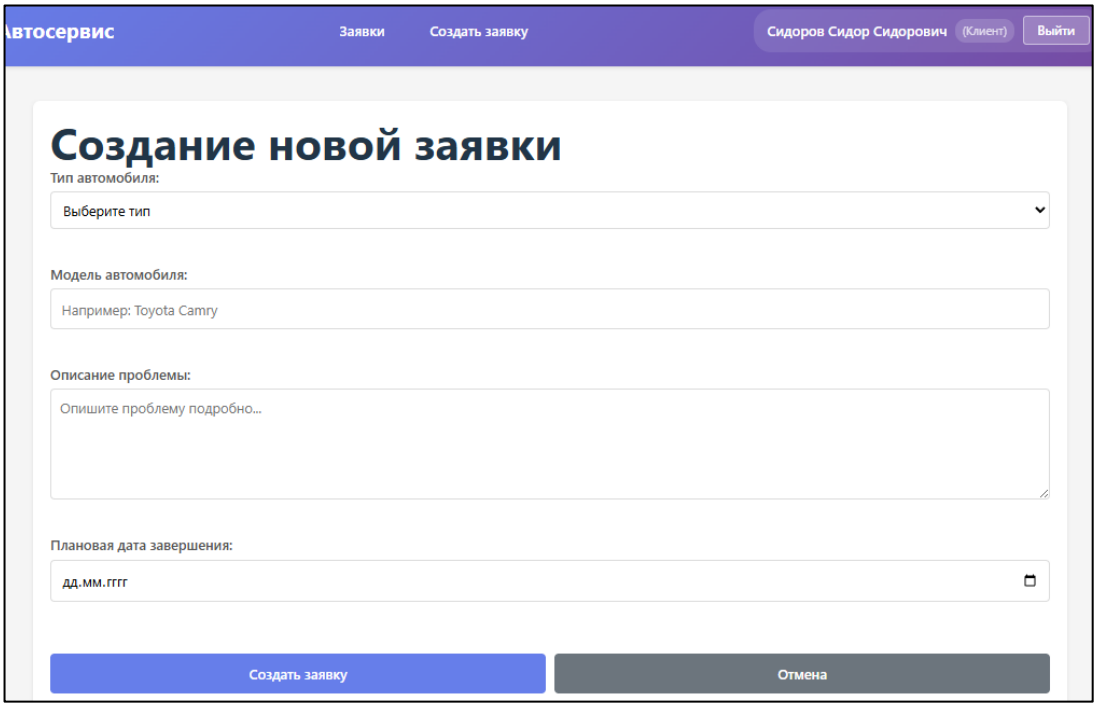

Рисунок 1.8 – Окно добавления новой заявки

На рисунке 1.9 представлена главная страница с заявками для пользователя «клиент». Имеется возможность просмотра собственных заявок, поиска по параметрам, просмотра.

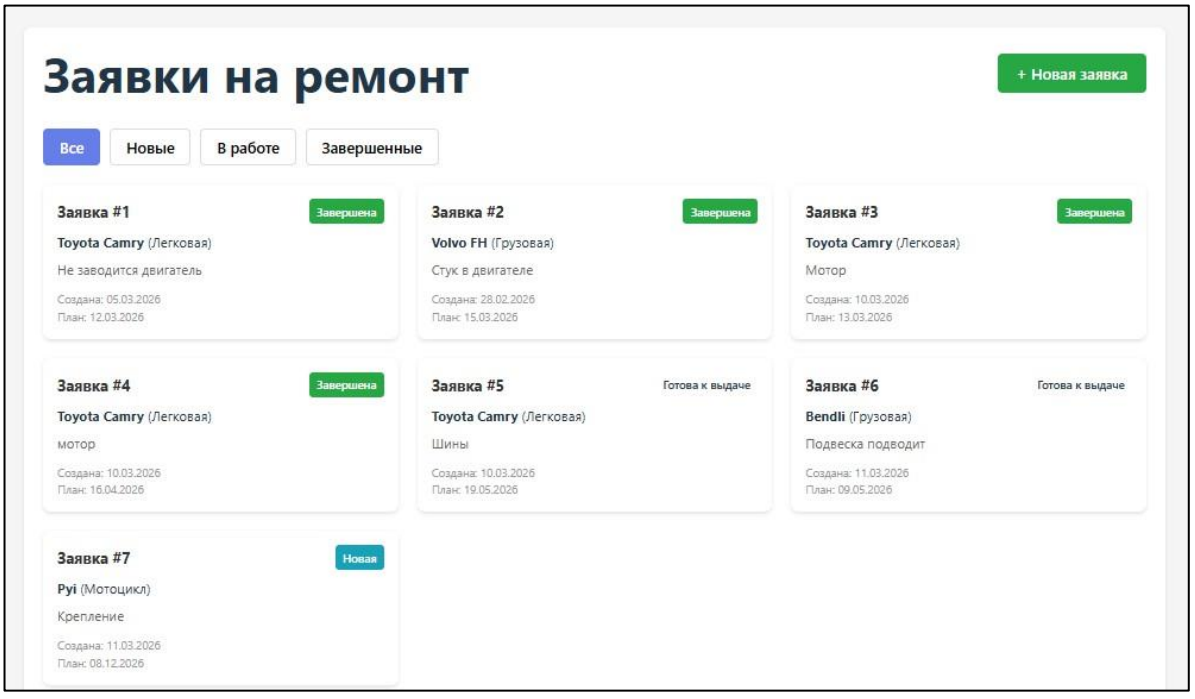

Рисунок 1.9– Главная страница

Окно добавления новой заявки представлено на рисунке 1.10 (для пользователя «клиент»). Имеются поля ввода типа автомобиля и представлена в виде динамического списка.

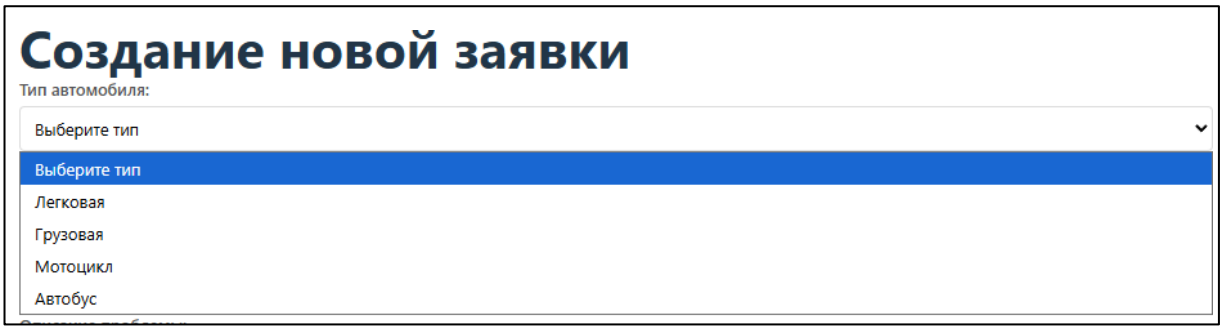

Рисунок 1.10 – Окно добавления заявки

На рисунке 1.11 представлено окно просмотра заявки (для пользователя «клиент»). Имеется возможность согласовать продление сроков ремонта и qr код на данную заявку.

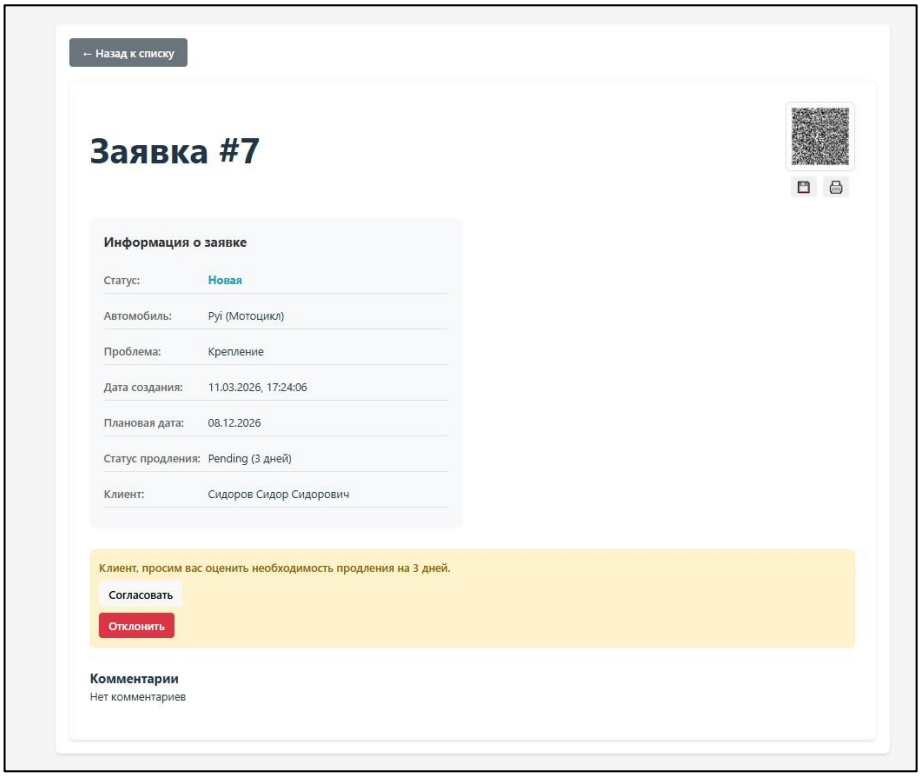

Рисунок 1.11 – Окно просмотра заявки

На рисунке 1.12 представлена страница с списком зарегистрированных аккаунтов в системе «админ». Имеется возможность просмотра собственных заявок, поиска по параметрам, просмотра и редактирования.

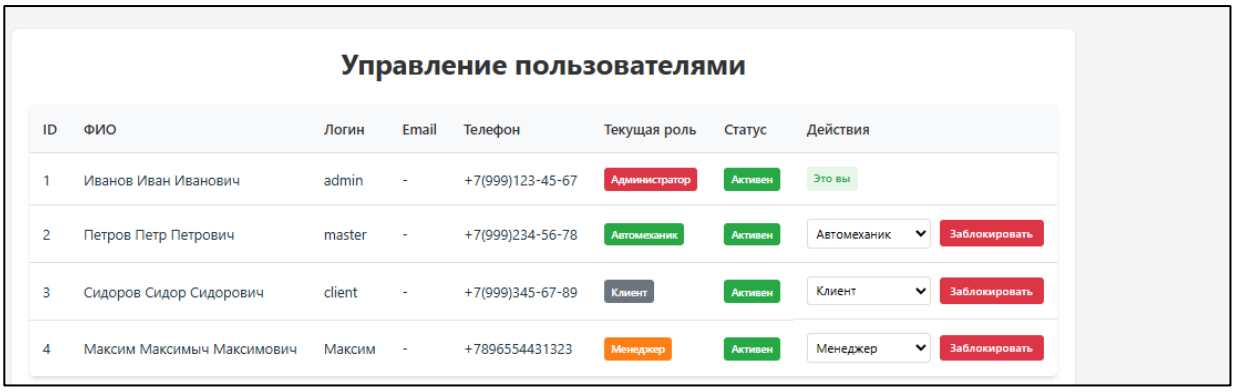

Рисунок 1.12 – Страница по управлению аккаунтов

На рисунке 1.13 представлено окно просмотра заявки (для пользователя «механик»). Имеется возможность просмотра всех деталей заявки, оставления комментариев и кнопка «Начать работу», но с автоматической добавление загруженности механика.

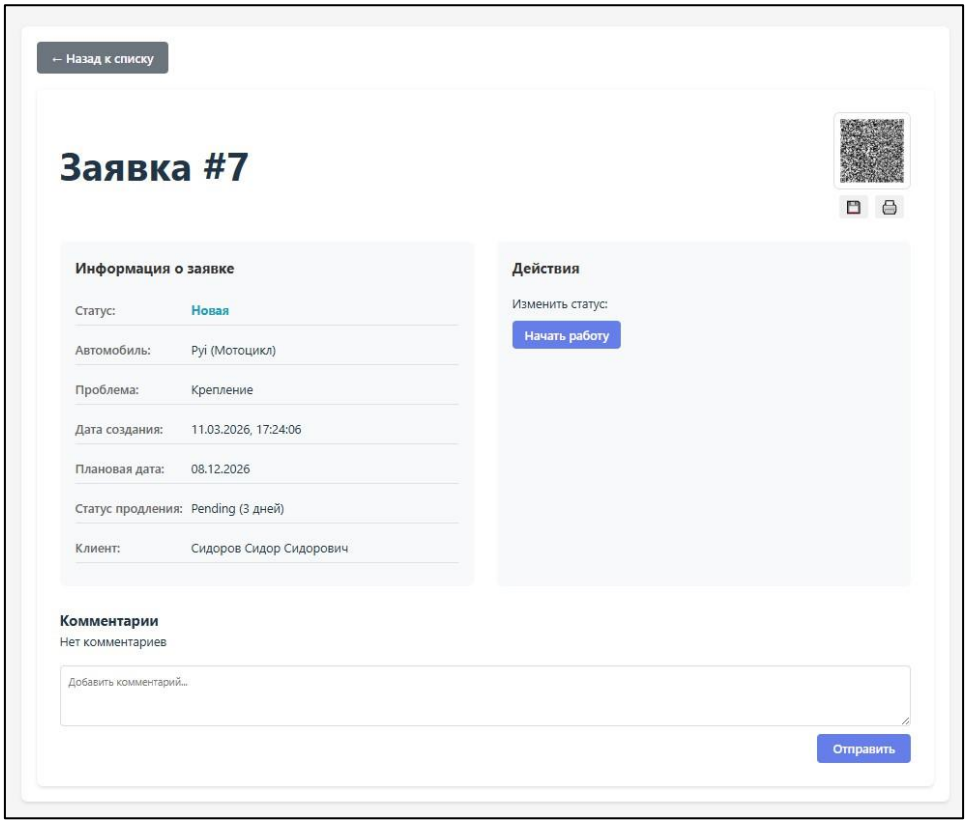

Рисунок 1.13 – Окно просмотра заявки

На рисунке 1.14 представлено окно qr кода на форму оценивание. Имеется возможность скачать или распечать qr код.

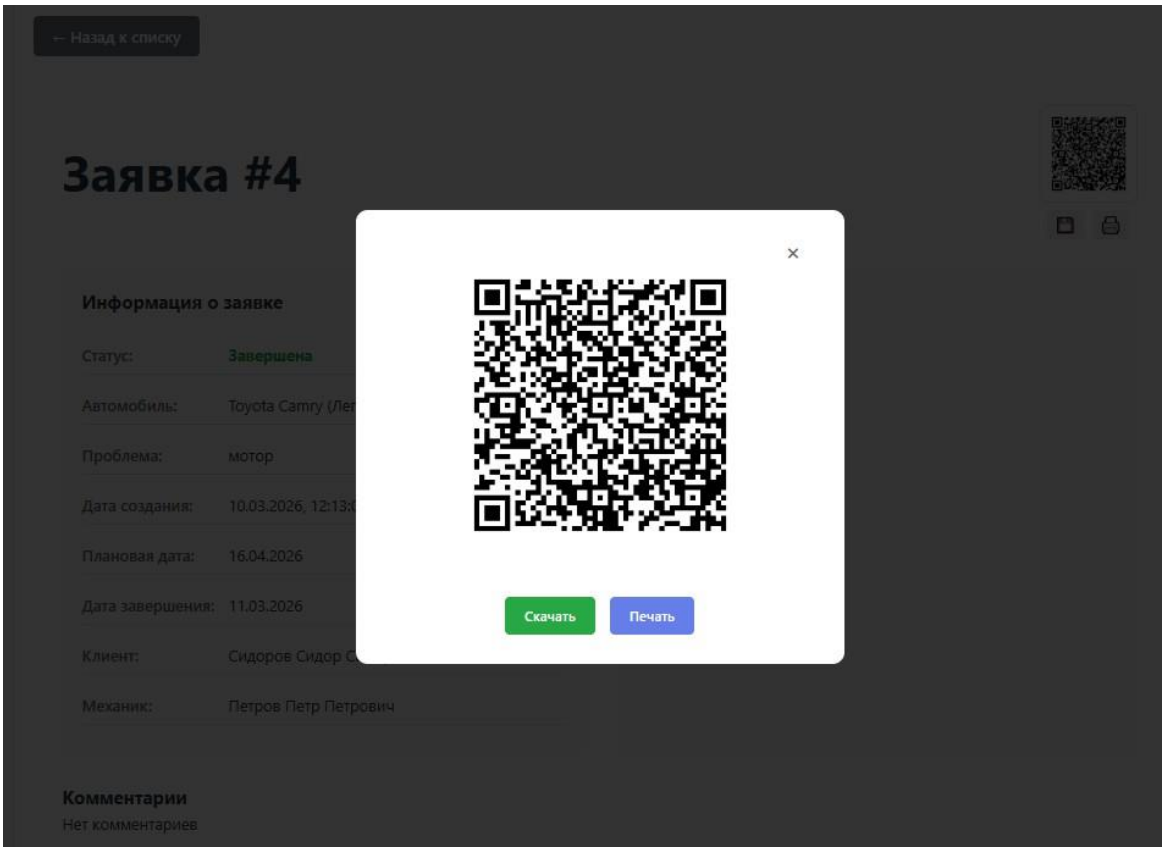

Рисунок 1.14 – Окно qr кода отсылающее на форму

2 Разработка функционала дипломной работы

2.1 Доработка модуля корзины покупок

Доработан модуль корзины покупок для корректного отображения данных о товарах, их количестве и стоимости. Исправлены ошибки, связанные с подсчетом уникальных товаров в индикаторе корзины и отображением информации о продуктах.На этапе разработки реализованы функции, позволяющие пользователю просматривать добавленные товары, изменять количество через инкремент/декремент, видеть итоговую стоимость заказа с учетом доставки и получать информацию о фермере, от которого заказаны товары. В настоящее время представлено рабочее приложение, демонстрирующее возможности данного модуля (Рисунок 3.1).

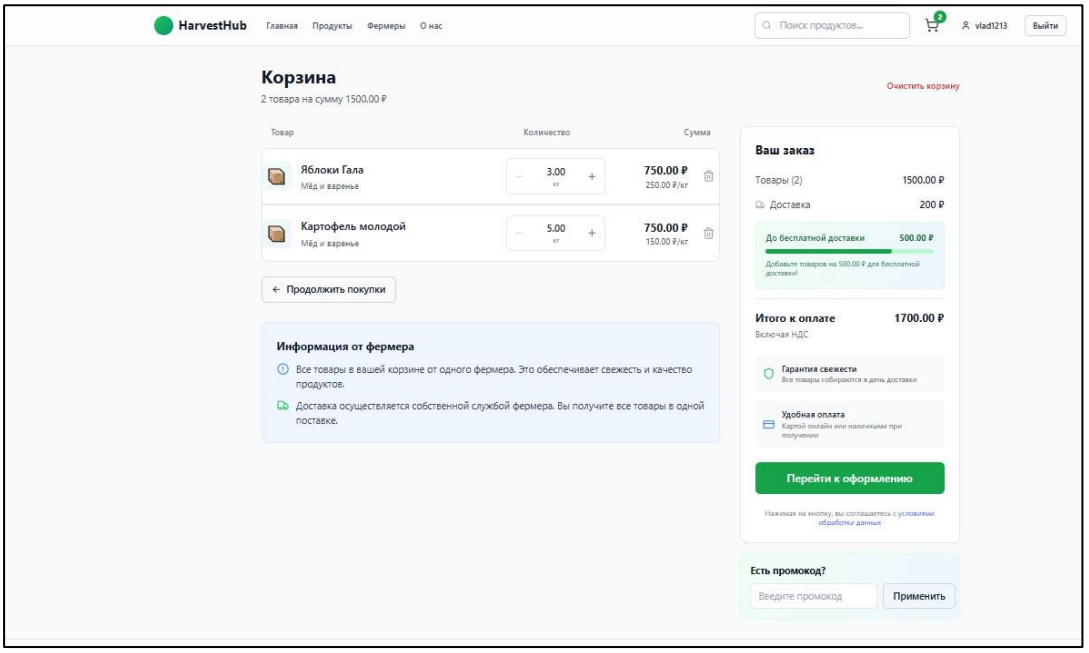

Рисунок 2.1 – Доработанный функциональность корзины

Инструменты, доступные пользователю, включают:

– просмотр списка товаров в корзине с указанием названия, категории, веса и цены за единицу измерения;

– изменение количества товаров через кнопки увеличения/уменьшения;

– автоматический пересчет итоговой суммы при изменении количества;

– отображение информации о доставке и расчет стоимости доставки;

– индикатор прогресса до бесплатной доставки;

– кнопку очистки корзины для удаления всех товаров;

масштабирования обеспечивает удобную настройку визуального представления, а выбор фигур, таких как прямоугольник, круг и линия, позволяет создавать разнообразные элементы модели. Кроме того, реализован выбор цвета как через основной набор, так и с использованием палитры для более точной настройки.

Дополнительные элементы интерфейса включают боковую панель с инструментами для быстрого доступа и рабочее поле, где пользователи могут создавать структурные схемы с иерархическими уровнями. Пример текущей формы демонстрирует базовые возможности программы, включая построение структурной модели с уровнями, соединительными линиями и текстовыми блоками, а также использование различных цветов, что наглядно иллюстрирует логику работы с инструментами.

В дальнейшем разработка будет сосредоточена на улучшении функциональности, внедрении более сложных опций редактирования и минимизации пользовательских ошибок. Основное внимание будет уделено созданию гибкого и удобного инструмента для проектирования структурных моделей различной сложности.

2.2 Разработка модуля отзывов о продуктах

Модуль отзывов реализован для управления обратной связью клиентов о продуктах (Рисунок 2.2).

Функционал предоставляет возможность оставлять отзывы о продуктах после покупки. В разделе создания отзыва пользователь может выбрать оценку от 1 до 5 звезд, добавить комментарий и отправить отзыв на модерацию. Отзывы отображаются на страницах продуктов и на страницах фермеров, что помогает другим покупателям принимать решения.

В интерфейсе приложения предусмотрен блок "Оставить отзыв", где пользователь может выбрать оценку через систему звезд, ввести комментарий (до 1000 символов) и отправить отзыв. После отправки отзыв проходит модерацию администратором перед публикацией. На странице продукта отображается средний рейтинг, общее количество отзывов и список опубликованных отзывов с указанием автора, даты и оценки.

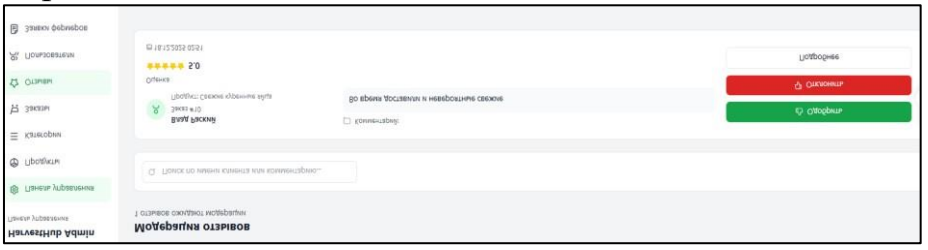

Рисунок 2.2 – Модуль отзывов

Для администраторов реализован раздел модерации отзывов, где можно просматривать ожидающие модерации отзывы, одобрять или отклонять их. Интерфейс интуитивен и структурирован, что делает процесс управления отзывами удобным и эффективным.

2.3 Разработка модуля фермерской панели и управления продукцией

Разработка модуля фермерской панели позволяет фермерам управлять своими продуктами и заказами (Рисунок 2.3).

Функционал фермерской панели предназначен для управления продукцией и обработки заказов. В интерфейсе приложения предусмотрены две основные вкладки: "Мои продукты" и "Мои заказы", которые обеспечивают полный контроль над деятельностью фермера на платформе.

В разделе "Мои продукты" фермер может:

просматривать список всех своих продуктов с информацией о цене, количестве, категории и статусе;

добавлять новые продукты через отдельную страницу с возможностью загрузки изображения;

редактировать существующие продукты;

удалять продукты из каталога.

Форма добавления продукта включает поля для названия, описания, цены за килограмм, количества в наличии, выбора категории, дат сбора урожая и срока годности, условий хранения, а также загрузку изображения продукта. Если изображение не загружено, автоматически используется заглушка с эмодзи, соответствующая категории продукта.

В разделе "Мои заказы" фермер может:

- просматривать все заказы на свои продукты;
- видеть статус каждого заказа (ожидает подтверждения, в обработке, ожидает указания веса, готов к отправке, отправлен, доставлен, завершен, отменен);
- изменять статус заказа через модальное окно;
- указывать реальный вес товаров для заказов со статусом "Ожидает указания веса";
- закрывать заказы после доставки, переводя их в статус "Завершен".

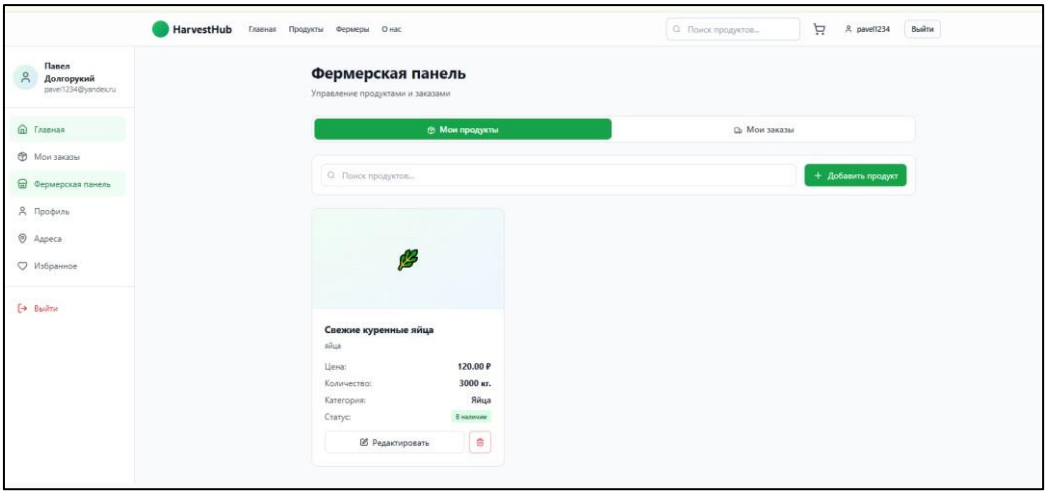

Рисунок 2.3 – Сохранение резервной копии

Дополнительно реализован процесс регистрации фермеров через систему заявок. Пользователи могут подать заявку на получение роли фермера, указав причину желания присоединиться к платформе. Администраторы могут рассматривать заявки, одобрять или отклонять их. При одобрении роль пользователя автоматически меняется на "Farmer", что открывает доступ к фермерской панели.

Интерфейс фермерской панели логично структурирован и обеспечивает удобное управление продукцией и заказами, что делает процесс работы фермера на платформе эффективным и прозрачным.

Заключение

В ходе учебной практики была выполнена доработка и расширение функциональности веб-приложения HarvestHub — платформы для торговли фермерскими продуктами. В рамках работы были реализованы три ключевых модуля, направленные на улучшение пользовательского опыта и расширение функциональных возможностей системы.

Основные результаты работы заключаются в доработке модуля корзины покупок — были исправлены ошибки отображения данных, реализован корректный подсчёт уникальных товаров в индикаторе корзины, а также улучшено представление информации о товарах с указанием веса, цены и категорий. Данный модуль обеспечивает удобное управление процессом покупок и корректный расчёт итоговой стоимости заказа.

В процессе разработки модуля отзывов о продуктах была реализована система обратной связи, позволяющая покупателям оставлять оценки и комментарии к товарам. Функциональность модуля включает модерацию отзывов администраторами, отображение среднего рейтинга и списка отзывов на страницах продуктов и фермеров, что способствует повышению прозрачности работы платформы, росту доверия со стороны покупателей и принятию более обоснованных решений при выборе продукции.

Разработка модуля фермерской панели позволила создать специализированный интерфейс для фермеров с возможностью управления ассортиментом и заказами. Реализованы функции добавления, редактирования и удаления продуктов с поддержкой загрузки изображений, обработки заказов с изменением их статусов, указанием фактического веса товаров и закрытием завершённых заказов. Дополнительно была внедрена система подачи заявок на получение роли фермера с последующим рассмотрением заявок администраторами платформы.

Все реализованные модули были интегрированы в существующую архитектуру приложения и протестированы на корректность работы, что подтверждает их соответствие требованиям современной веб-разработки. В процессе реализации использовались технологии React с TypeScript для фронтенд-части и ASP.NET Core с Entity Framework Core для серверной части, что обеспечивает надёжность, производительность и масштабируемость разработанного решения.

Практическая значимость выполненной работы заключается в повышении функциональности платформы, улучшении взаимодействия между покупателями и фермерами, а также в автоматизации ключевых бизнес-процессов и повышении удобства использования системы. Реализованный модуль отзывов способствует повышению прозрачности и доверия к платформе, фермерская панель упрощает управление продукцией и заказами, а доработанный модуль корзины обеспечивает корректную и стабильную работу процесса оформления покупок.

Библиография

1 React – официальная документация [Электронный ресурс] – Режим доступа: https://react.dev/;
2 TypeScript – официальная документация [Электронный ресурс] – Режим доступа: https://www.typescriptlang.org/docs/;
3 ASP.NET Core – официальная документация [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/aspnet/core/;
4 Entity Framework Core – официальная документация [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/ef/core/;
5 PostgreSQL – официальная документация [Электронный ресурс] – Режим доступа: https://www.postgresql.org/docs/;
6 Zustand – документация по управлению состоянием [Электронный ресурс] – Режим доступа: https://docs.pmnd.rs/zustand/getting-started/introduction;
7 React Router – официальная документация [Электронный ресурс] – Режим доступа: https://reactrouter.com/;
8 Vite – официальная документация [Электронный ресурс] – Режим доступа: https://vitejs.dev/;
9 Tailwind CSS – официальная документация [Электронный ресурс] – Режим доступа: https://tailwindcss.com/docs;
10 RESTful API Design – Best Practices [Электронный ресурс] – Режим доступа: https://restfulapi.net/;
11 React Hook Form – документация [Электронный ресурс] – Режим доступа: https://react-hook-form.com/;
12 Axios – документация HTTP клиента [Электронный ресурс] – Режим доступа: https://axios-http.com/docs/intro;
13 Lucide React – библиотека иконок [Электронный ресурс] – Режим доступа: https://lucide.dev/;
14 React Hot Toast – документация уведомлений [Электронный ресурс] – Режим доступа: https://react-hot-toast.com/;
15 date-fns – документация работы с датами [Электронный ресурс] – Режим доступа: https://date-fns.org/;
16 C# Programming Guide [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/csharp/;
17 PostgreSQL и Entity Framework Core – интеграция [Электронный ресурс] – Режим доступа: https://www.npgsql.org/efcore/;
18 JWT Authentication в ASP.NET Core [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/aspnet/core/security/authentication/jwt-authn;
19 React TypeScript Cheatsheet [Электронный ресурс] – Режим доступа: https://react-typescript-cheatsheet.netlify.app/;
20 Web Development Best Practices [Электронный ресурс] – Режим доступа: https://developer.mozilla.org/ru/docs/Web/Guide;

Приложение А
(справочное)
Листинг программы

```
mport api from './axios'
import { ApiResponse } from '@/types/backend'

export interface FarmerApplicationDto {
    applicationId: number
    userId: number
    userEmail: string
    userName: string
    message: string
    status: string
    adminNotes?: string
    reviewedBy?: number
    reviewerName?: string
    createdAt: string
    reviewedAt?: string
}

export interface CreateFarmerApplicationDto {
    message: string
}

export interface ReviewFarmerApplicationDto {
    status: string // 'Approved' | 'Rejected'
    adminNotes?: string
}

export const farmerApplicationsApi = {
    createApplication: async (data: CreateFarmerApplicationDto):
Promise<FarmerApplicationDto> => {
        const response = await
api.post<ApiResponse<FarmerApplicationDto>>('/FarmerApplications
', data)
        if (!response.data.success) {
            throw new Error(response.data.message || 'Failed to
create application')
        }
        return response.data.data
    },
```

```
  getMyApplication: async (): Promise<FarmerApplicationDto | null>
 => {
        try {
            const response = await
api.get<ApiResponse<FarmerApplicationDto>>('/FarmerApplications/my')
            if (!response.data.success) {
                return null
            }
            return response.data.data
        } catch (error: any) {
            if (error.response?.status === 404) {
                return null
            }
            throw error
        }
    },

    checkPendingApplication: async (): Promise<boolean> => {
        const response = await
api.get<ApiResponse<boolean>>('/FarmerApplications/check')
        if (!response.data.success) {
            throw new Error(response.data.message || 'Failed to check
application')
        }
        return response.data.data
    },

    getAllApplications: async (): Promise<FarmerApplicationDto[]>
=> {
        const response = await
api.get<ApiResponse<FarmerApplicationDto[]>>('/FarmerApplications/all'
)
        if (!response.data.success) {
            throw new Error(response.data.message || 'Failed to fetch
applications')
        }
        return response.data.data
    },

    getPendingApplications: async (): Promise<FarmerApplicationDto[]>
=> {
        const response = await
api.get<ApiResponse<FarmerApplicationDto[]>>('/FarmerApplications/peni
ng')
        if (!response.data.success) {
            throw new Error(response.data.message || 'Failed to fetch
pending applications')
        }
        return response.data.data
    },
```

```
 getApplication: async (id: number): Promise<FarmerApplicationDto> => {
        const response = await
api.get<ApiResponse<FarmerApplicationDto>>(`/FarmerApplications/${id}`)
        if (!response.data.success) {
            throw new Error(response.data.message || 'Failed to fetch
application')
        }
        return response.data.data
    },

    reviewApplication: async (id: number, data:
ReviewFarmerApplicationDto): Promise<FarmerApplicationDto> => {
        const response = await
api.put<ApiResponse<FarmerApplicationDto>>(`/FarmerApplications/${id}/r
eview`, data)
        if (!response.data.success) {
            throw new Error(response.data.message || 'Failed to review
application')
        }
        return response.data.data
    }
}
```

```
import { useEffect, useState } from 'react'
import { Link, useNavigate } from 'react-router-dom'
import { farmersApi, FarmerDto } from '@/api/farmers.api'
import { farmerApplicationsApi, FarmerApplicationDto,
CreateFarmerApplicationDto } from '@/api/farmerApplications.api'
import { useAuthStore } from '@/store/auth.store'
import { MapPin, Package, Loader2, X, CheckCircle, Clock } from
'lucide-react'

export default function FarmersPage() {
    const { user, isAuthenticated } = useAuthStore()
const { user, isAuthenticated } = useAuthStore()
    const navigate = useNavigate()
    const [farmers, setFarmers] = useState<FarmerDto[]>([])
    const [isLoading, setIsLoading] = useState(true)
    const [error, setError] = useState<string | null>(null)
    const [isApplicationModalOpen, setIsApplicationModalOpen] =
useState(false)
    const [applicationMessage, setApplicationMessage] = useState('')
    const [isSubmitting, setIsSubmitting] = useState(false)
    const [myApplication, setMyApplication] =
useState<FarmerApplicationDto | null>(null)
    const [isCheckingApplication, setIsCheckingApplication] =
useState(false)

    useEffect(() => {
        const loadFarmers = async () => {
            setIsLoading(true)
            setError(null)
            try {
                const fetchedFarmers = await
farmersApi.getAllFarmers()
                setFarmers(fetchedFarmers)
            } catch (err) {
                console.error('Failed to load farmers:', err)
                setError(err instanceof Error ? err.message : 'Не
удалось загрузить фермеров')
            } finally {
                setIsLoading(false)
            }
        }

        loadFarmers()
    }, [])
```

```
 useEffect(() => {
        const checkMyApplication = async () => {
            if (isAuthenticated && user?.role ===
'Customer') {
                setIsCheckingApplication(true)
                try {
                    const application = await
farmerApplicationsApi.getMyApplication()
                    setMyApplication(application)
                } catch (err) {
                    console.error('Failed to check
application:', err)
                } finally {
                    setIsCheckingApplication(false)
                }
            }
        }

        checkMyApplication()
    }, [isAuthenticated, user])

    const getFarmerEmoji = (index: number) => {
        const emojis = ['🌾', '🚜', '🌿', '🥔', '🥕',
'🍎', '□', '🍉']
        return emojis[index % emojis.length]
    }

    const getFarmerColor = (index: number) => {
        const colors = ['bg-green-200', 'bg-blue-200',
'bg-yellow-200', 'bg-red-200', 'bg-purple-200', 'bg-
pink-200']
        return colors[index % colors.length]
    }

    const handleOpenApplicationModal = () => {
        if (!isAuthenticated) {
            navigate('/login?redirect=/farmers')
            return
        }
        if (user?.role !== 'Customer') {
            alert('Только пользователи могут подавать
заявку на роль фермера')
            return
        }
        setIsApplicationModalOpen(true)
    }

    const handleSubmitApplication = async () => {
        if (!applicationMessage.trim() ||
applicationMessage.trim().length < 10) {
            alert('Сообщение должно содержать минимум 10
символов')
        }
```

```
return
        }

        setIsSubmitting(true)
        try {
            const data: CreateFarmerApplicationDto = {
                message: applicationMessage.trim()
            }
            const application = await farmerApplicationsApi.createApplication(data)
            setMyApplication(application)
            setIsApplicationModalOpen(false)
            setApplicationMessage('')
            alert('Заявка успешно подана! Администратор рассмотрит её в ближайшее
время.')
        } catch (err) {
            console.error('Failed to submit application:', err)
            alert(err instanceof Error ? err.message : 'Не удалось подать заявку')
        } finally {
            setIsSubmitting(false)
        }
    }

    const getApplicationStatusBadge = () => {
        if (!myApplication) return null

        if (myApplication.status === 'Pending') {
            return (
                <div className="flex items-center gap-2 text-yellow-600 bg-yellow-50 px-3 py-1
rounded-full text-sm">
                    <Clock className="h-4 w-4" />
                    <span>Заявка на рассмотрении</span>
                </div>
            )
        }

        if (myApplication.status === 'Approved') {
            return (
                <div className="flex items-center gap-2 text-green-600 bg-green-50 px-3 py-1 rounded-
full text-sm">
                    <CheckCircle className="h-4 w-4" />
                    <span>Заявка одобрена</span>
                </div>
            )
        }
```

```
if (myApplication.status === 'Rejected') {
            return (
                <div *className*="flex items-center gap-2 text-red-600
bg-red-50 px-3 py-1 rounded-full text-sm">
                    <X *className*="h-4 w-4" />
                    <span>Заявка отклонена</span>
                </div>
            )
        }

        return null
    }

    if (isLoading) {
        return (
            <div *className*="container mx-auto px-4 py-8">
                <div *className*="flex items-center justify-center h-
64">
                    <div *className*="text-center">
                        <Loader2 *className*="h-12 w-12 animate-spin
text-green-600 mx-auto mb-4" />
                        <p *className*="text-gray-600">Загрузка
фермеров...</p>
                    </div>
                </div>
            </div>
        )
    }

    if (error) {
        return (
            <div *className*="container mx-auto px-4 py-8">
                <div *className*="text-center py-12">
                    <div *className*="text-4xl mb-2">😕</div>
                    <h1 *className*="text-xl font-bold mb-2">Ошибка
загрузки</h1>
                    <p *className*="text-gray-600 mb-6">{error}</p>
                    <button
                        *onClick*={() => window.location.reload()}
                        *className*="px-6 py-2 bg-green-600 text-white
rounded-lg hover:bg-green-700"
                    >
                        Попробовать снова
                    </button>
                </div>
            </div>
        )
    }
```

```
 {farmers.length === 0 ? (
                <div  className="text-center py-12">
                    <div  className="text-4xl mb-2">🌾</div>
                    <h2  className="text-xl  font-bold  mb-2">Фермеров
пока нет</h2>
                        <p  className="text-gray-600">Станьте первым
фермером на нашей платформе!</p>
                </div>
            ) : (
                <div  className="grid md:grid-cols-2 gap-6">
                    {farmers.map((farmer, index) => (
                        <Link
                            key={farmer.farmerId}
                            to={`/farmers/${farmer.farmerId}`}
                             className="border rounded-lg overflow-
hidden shadow-sm hover:shadow-md transition-shadow"
                        >
                                                <div  className={`h-48
${getFarmerColor(index)} flex items-center justify-center`}>
                                                <div  className="text-
 4xl">{getFarmerEmoji(index)}</div>
                            </div>
                            <div  className="p-6">
                                      <h3  className="text-xl  font-
semibold mb-2">
                                                {farmer.firstName}
{farmer.lastName}
                                  </h3>
                                    <div  className="flex  items-center
gap-4 mb-3 flex-wrap">
                                      {farmer.specialty && (
                                           <span  className="px-3 py-1
bg-gray-100 text-gray-800 rounded-full text-sm">
                                                {farmer.specialty}
                                          </span>
                                      )}
                                      {farmer.address && (
                                          <span  className="text-gray-
600 text-sm flex items-center gap-1">
                                                <MapPin  className="h-4
w-4" />
                                                {farmer.address}
                                          </span>
                                      )}
                                  </div>
                                  {farmer.description && (
                                       <p  className="text-gray-700 mb-
4">{farmer.description}</p>
                                  )}
                                    <div  className="flex  items-center
gap-2 text-sm text-gray-600 mb-4">
                                      <Package  className="h-4 w-4" />
```

```
 <span>{farmer.productCount} продукт{farmer.productCount !== 1 ? 'ов'
: ''}</span>
                                    </div>
                                    <div  className="text-green-600
hover:text-green-700 font-medium">
                                        Посмотреть продукты →
                                    </div>
                                </div>
                            </Link>
                        ))}
                    </div>
                )}
<div  className="mt-12 bg-gradient-to-r from-green-50 to-blue-50 p-8
rounded-lg">
                    <h2  className="text-2xl font-bold mb-4">Станьте нашим
фермером</h2>
                    <p  className="text-gray-700 mb-6">
                        Если вы производите качественные фермерские
продукты и хотите присоединиться к нашей платформе,
                        мы будем рады сотрудничеству!
                    </p>
{isAuthenticated && user?.role === 'Customer' && (
                        <div  className="mb-4">
                            {getApplicationStatusBadge()}
                        </div>
                    )}
                    {isAuthenticated && user?.role === 'Customer' &&
!myApplication && (
                        <button
                            onClick={handleOpenApplicationModal}
                            className="bg-green-600 text-white px-6 py-3
rounded-lg hover:bg-green-700 font-medium transition-colors"
                        >
                            Подать заявку
                        </button>
                    )}
                    {!isAuthenticated && (
                        <button
                            onClick={() =>
navigate('/login?redirect=/farmers')}
                            className="bg-green-600 text-white px-6 py-3
rounded-lg hover:bg-green-700 font-medium transition-colors"
                        >
                            Войти, чтобы подать заявку
                        </button>
                    )}
```

```
 )}
                {(user?.role === 'Farmer' || user?.role === 'Admin') &&
(
                    <p class="text-gray-600 italic">
                        Вы уже являетесь {user.role === 'Farmer' ?
'фермером' : 'администратором'}
                    </p>
                )}
            </div>

            {/* Модал для подачи заявки */}
            {isApplicationModalOpen && (
                <div class="fixed inset-0 bg-black bg-opacity-50
flex items-center justify-center p-4 z-50">
                    <div class="bg-white rounded-xl w-full max-w-md
shadow-lg">
                        <div class="p-6">
                            <div class="flex items-center justify-
between mb-6">
                                <h2 class="text-xl font-
bold">Подать заявку на роль фермера</h2>
                                <button
                                    onClick={() =>
setIsApplicationModalOpen(false)}
                                    class="p-2 rounded-full
hover:bg-gray-100 transition-colors"
                                    disabled={isSubmitting}
                                >
                                    <X class="h-6 w-6 text-gray-
600" />
                                </button>
                            </div>

                            <div class="mb-4">
                                <label class="block text-sm font-
medium text-gray-700 mb-2">
                                    Расскажите о себе и вашей продукции
*
                                </label>
                                <textarea
                                    value={applicationMessage}
                                    onChange={(e) =>
setApplicationMessage(e.target.value)}
                                    placeholder="Минимум 10 символов.
Расскажите о себе, вашем опыте, продукции и почему вы хотите стать
фермером на нашей платформе..."
                                    class="w-full border rounded-lg
px-3 py-2 focus:ring-2 focus:ring-green-500 focus:border-transparent
min-h-[120px] resize-y"
                                    disabled={isSubmitting}
                                    maxLength={500}
                                />
```

```
  <p class="text-xs text-gray-500 mt-1">
                                    {applicationMessage.length}/500
символов
                                </p>
                            </div>

                            <div class="flex justify-end gap-3">
                                <button
                                    onClick={() =>
setIsApplicationModalOpen(false)}
                                    class="px-4 py-2 border border-
gray-300 rounded-md text-gray-700 hover:bg-gray-50 transition-colors"
                                    disabled={isSubmitting}
                                >
                                    Отмена
                                </button>
                                <button
                                    onClick={handleSubmitApplication}
                                    disabled={isSubmitting ||
applicationMessage.trim().length < 10}
                                    class="px-4 py-2 bg-green-600
text-white rounded-md hover:bg-green-700 transition-colors
disabled:opacity-50 disabled:cursor-not-allowed"
                                >
                                    {isSubmitting ? (
                                        <>
                                            <Loader2 class="h-4 w-4
animate-spin inline mr-2" />
                                            Отправка...
                                        </>
                                    ) : (
                                        'Подать заявку'
                                    )}
                                </button>
                            </div>
                        </div>
                    </div>
                </div>
            )}
        </div>
    )
}
```